

Crop Yield Prediction & Crop Recommendation System

Course: Fundamentals of AI and Machine Learning

Submission: VITyarthi — Build Your Own Project

Student Name: POLISETTY HARSHA SIVARAMA KUMAR GUPTHA

Registration Number: 25MIM10039

Programme / Branch: INTEGRATED MTECH AI

Institution :VIT BHOPAL UNIVERSITY

GitHub Repository: <https://github.com/polisetty25mim10039-lang/Crop-Yield-Prediction-Crop-Recommendation-System>

Table of Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-functional Requirements
5. System Architecture
6. Design Diagrams
7. Design Decisions & Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Learnings & Key Takeaways
13. Future Enhancements
14. References

1.Introduction

Agriculture in India involves a lot of people. At the same time, when making decisions about the type of cultivation and expected yields depending on the crop choice for a particular piece of land, experience and heuristics are the main sources of information. Every year there arise two questions: which crops would be best to grow on a particular piece of land and how much could be harvested from it? Ultimately, the questions require pattern recognition from historical data, and thus supervised machine learning could solve them.

In the current implementation, there is an AI/ML command-line tool that provides answers to those questions using data instead of intuition. The implementation uses two separate supervised machine learning models:

- A classifier that determines what crops should be planted in a plot depending on its soil composition (nitrogen, phosphorus, potassium, pH) and forecasted weather conditions (temperature, humidity, precipitation).
- A regressor that predicts yield in tonnes per hectare and therefore calculates total production of a certain crop in a particular state, at a specific season and plot area.

This implementation represents the full lifecycle of a machine learning project including data input and validation, data cleansing and outliers elimination, original feature engineering, model training using exchangeable algorithms, metrics, model persistence, validated inference via a user interface, centralized logging, and unit testing. Every step is represented in a separate module to show the architecture of the system in code.

The implementation is created in Python 3 with pandas and NumPy, scikit-learn, joblib, PyYAML, and pytest libraries. The system runs using a command line interface without any graphical user interface elements or network access during inference.

2. Problem Statement

2.1 The Problem

At each growing cycle, there are two interrelated decisions for a farmer or an agriculture professional to make. First, there is a problem of crop selection: a piece of land has its own unique composition of nutrients and pH levels, while the next season is predicted to have its unique temperature, humidity and precipitation characteristics. Performance of crops varies under these conditions: some crops require nitrogen-based, balanced nutrient composition, others – potassium-biased; some crops tolerate high levels of precipitation, others – don't. An incorrect crop selection leads to bad germination, extra costs required to compensate for the negative impact and decreased yields. The right selection implies the need to take into account simultaneously seven variables, which is impossible to do using only human intuition.

The second decision-making process is that of estimation. After a crop is selected, a farmer needs to make plans on expenditure on seeds and fertilizers, arrange transportation and storage, estimate his future income based on the results of harvesting. Crop yield varies systematically depending on the selected crop, state (as a proxy for soil type, existence of irrigation and climate), season and scale of cultivation. The rules can be found in the historical crop production records but they cannot be used by the individual planer.

Now both problems are solved using empirical approaches. Such approaches usually work badly in case of a start of a new season, introduction of a new piece of land and planting of an unfamiliar crop. There is a possibility to create a data-based system, which would learn these rules from historical data.

2.2 Objectives

This project will strive to achieve the following objectives:

1. Consume and validate two separate real public agricultural data sets with different structures.
2. Create reproducible preprocessing workflows for missing value handling, correction of data entry outliers, creation of the target variable for modelling when it is not readily available and creation of other predictive variables.
3. Invent at least one new engineered variable per each data set instead of using raw variables as model input directly.
4. Train one classifier and one regressor; the specific algorithm is configurable so that different algorithms could be evaluated without changing any code.
5. Evaluate both models with metrics suited to each task separately.
6. Save the trained models and enable both capabilities in one validated command line interface that produces machine readable output.

7. Adhere to the engineering quality criteria: modular architecture, externalized configuration, centralized logging, input validation and automated unit tests.

2.3 Scope

Within scope: ingestion of two structured CSV datasets; preprocessing, target extraction and feature engineering;

training and evaluation of a classifier for crop recommendation and a regressor for yield prediction; model

persistence on disk; a two-command CLI for inference with strict input validation and JSON output; logging of

all requests and responses; unit tests over preprocessing, training and validation code.

Not within scope: real-time ingestion of weather or satellite data; user interface for the web and/or mobile apps; multiregional or long-term forecasting for several years ahead; economic optimisation (maximisation of profit rather

than yield). All of these are elaborated on in Section 13 as future enhancements.

2.4 Target Users

Farmers themselves and smallholder farmers making a decision on which crops to grow next season on the

basis of the soil test result for their plots and predicted weather.

Agricultural extension agents and specialists answering questions from farmers about the choice of the

next-season crops.

Agri-analysis specialists in regional or statewide agencies calculating the aggregate production based on

the analysis above.

3. Functional Requirements

The software system includes 3 main modules of functioning. Each one shows a separate stage of the pipeline and also a separate set of files.

Module 1 – : Data Loading and Preprocessing

- 1.1 Load both source datasets from paths specified in config.yaml.
- 1.2 Do structural validation on load: file exists, file is not empty, all columns required are present. Raise a specific exception `LoaderError` otherwise.
- 1.3 Normalise whitespace in categorical text columns due to real exports of production dataset having padded values such as "Kharif".
- 1.4 Coerce numeric columns to numerical type and replace NA values in them with the column median.
- 1.5 Clip outlier values, applying a 4 standard deviations window for soil/weather features and a 6 upper bound standard deviations for derived features.
- 1.6 Derive the regression target `yield_tons_per_ha = Production / Area`, discarding records where area or production values are null or area value is zero.
- 1.7 Derive the engineered feature `npk_balance_score` for the recommendation dataset.
- 1.8 Derive the engineered feature `area_percentile` for the production dataset.
- 1.9 Label encode all categorical columns and return the fit encoder for further reuse during prediction stage.
- 1.10 Save processed datasets to `data/processed/` for review and reproducibility.

Module 2:- Model Training and Evaluation

- 2.1 Split both datasets into training and test subsets according to the ratio and seed from config.yaml, stratify the classification split by crop label.
- 2.2 Train a classifier on the recommendation dataset to predict crop label.
- 2.3 Train a regressor on the production dataset to predict yield per hectare.
- 2.4 Select concrete algorithms for both tasks according to configuration, support Random Forest and Gradient Boosting variants.
- 2.5 Assess classifier performance using accuracy, weighted precision, weighted recall and weighted F1.
- 2.6 Assess regressor performance using RMSE, MAE and R^2 .
- 2.7 Save both trained models and both encoder sets to `models/` using joblib.
- 2.8 Re-train the whole system from a single command (`python train.py`) and print the metrics summary.

Module 3:- Inference / Prediction

3.1 Define a recommend-crop subcommand taking N, P, K, temperature, humidity, pH, rainfall and an optional top-N number.

3.2 Return the top-N crops ordered by predicted suitability probability.

3.3 Define a predict-yield subcommand taking state, season, crop, area and an optional crop year.

3.4 Return predicted yield per hectare and total production for the provided area.

3.5 Validate all numeric inputs against a physically meaningful range before passing them to the model.

3.6 Validate all categorical inputs against the categories that the model was trained on case-insensitive and provide the allowed values in the error message.

3.7 Output the results as an indented JSON on standard output, ready to be parsed in scripts.

3.8 Return 0 exit code in case of success and non-zero exit code with a message on standard error in case of failure.

3.9 Log each request and its outcome, with timestamp, to logs/app.log.

3.1 Input / Output Structure

recommend-crop

Inputs: --n, --p, --k (nutrient levels), --temperature (°C), --humidity (%), --ph, --rainfall (mm), --top-n (default

3).

Output: JSON object consisting of a recommendations array of {crop, suitability} pairs sorted by descending

suitability.

predict-yield

Inputs

```
(venv) PS C:\Users\harsh\Downloads\crop-yield-predictor-trained\crop-yield-predictor> python main.py predict-yield --state Punjab --season Kharif --crop Rice --area 2.5
```

Output:

```
{
  "crop": "Rice",
  "state": "Punjab",
  "season": "Kharif",
  "predicted_yield_tons_per_ha": 2.631,
  "predicted_production_tons": 6.577,
  "area_ha": 2.5
}
```

3.2 User Workflow

1. The user installs dependencies from requirements.txt and provides two raw datasets in data/raw/ .

2. The user runs `python train.py`. Both pipelines run: ingestion, preprocessing, feature engineering, training,

evaluation and the four model artefacts are saved in `models/`. Summary of metrics is output to the terminal.

3. To produce a plot based on known soil and weather conditions, the user runs `recommend-crop` and gets a

ranked list of suitable crops.

4. Having chosen one of the listed crops, the user runs `predict-yield` providing their state, season and plot area,

and gets expected yield and total production figures.

5. Any invalid input at any of the stages produces a specific error message, pointing out the problematic input and

accepted range or value set – no silent coercions.

6. All requests and results are logged in `logs/app.log`

4. Non-functional Requirements

Performance :Inference must be interactive. Training happens once; at query time the models are loaded from disk and never retrained on a prediction. A single CLI query takes well less than a second after model load. Preprocessing and feature engineering uses only vectorised pandas and NumPy functions (`groupby(...).rank(pct=True)` for the area percentile, row-wise aggregation for the NPK balance score) with no loop in Python code, so inference and training on the entire 246,099-row production dataset takes minutes.

Reliability:The system must never return a confidently wrong result when given meaningless input. All numeric CLI arguments are checked to fall within the allowed ranges (pH: 0–14, humidity: 0–100, temperature: –10–55 °C, rainfall: 0–1,000 mm, area: 0.01–1,000 ha, crop year: 1990–2035) and all categorical arguments are checked against the classes in the encoder learned from the training data. Any failure raises `ValidationError` and aborts before running the model.

Maintainability: None of the paths, hyperparameters or logging levels is hard-coded in the source code; everything goes to `config.yaml`. Single-purpose responsibilities are implemented in individual modules under `src/`, with a docstring describing which functional module it implements. Estimator algorithm selection is based on data, so introduction of a new estimator only requires adding another entry to the registry, without modifying the training logic.

Error Handling Strategy: Different kinds of error are processed separately rather than using a single catch-all. Configuration errors raise `ConfigError`; dataset errors with a name of the column that is missing or absent from the schema raise `DataLoaderError`; input errors raise `ValidationError`; lack of model artifacts raises a prompt to run `train.py` first. The CLI reports each kind of failure with a descriptive message on standard error stream and a non-zero exit code. An unexpected failure is logged with a full trace to the log file and reported back to the user as a concise message without stack trace in the user interface.

Logging & Monitoring :A single `get_logger()` function in `src/logger.py` timestamps, names and logs every message at the right level to both the console and `logs/app.log`, preventing double registration

of the handler. Row counts for datasets, number of samples for training, number of features, evaluation metrics, inference requests and their parameters, as well as validation errors are all logged, providing a complete audit trail of what the system was asked and how it responded.

Scalability: Vectorisation of the pipeline and disk-based persistence of the model makes the same code work on an order of magnitude bigger dataset without modifications. The `src/` modules are importable and CLI-free, so the same logic can be served through a web API later without having to change anything.

Usability: The interface is fully self-documenting: `--help` output for the top-level command and for each subcommand lists every parameter with its canonical form and rejection rules. The validation error messages provide a list of all the valid values rather than just reporting the failure. Indented JSON output is both human-readable and easily parseable.

Reproducibility: A fixed random seed (42) from configuration determines both the train-test split and estimator initialization, ensuring the same models and metrics on different training runs. The cleaned intermediate datasets are stored in `data/processed/` allowing the reproduction of any reported result from the precise dataset.

5. System Architecture

The system follows a layered pipeline architecture with two parallel data paths that converge at a shared interface layer. The layers are:

Layer 1 — Data Layer. Two raw CSV datasets in `data/raw/`, the cleaned outputs in `data/processed/`, and the serialised model and encoder artefacts in `models/`. This layer holds state only; it contains no logic.

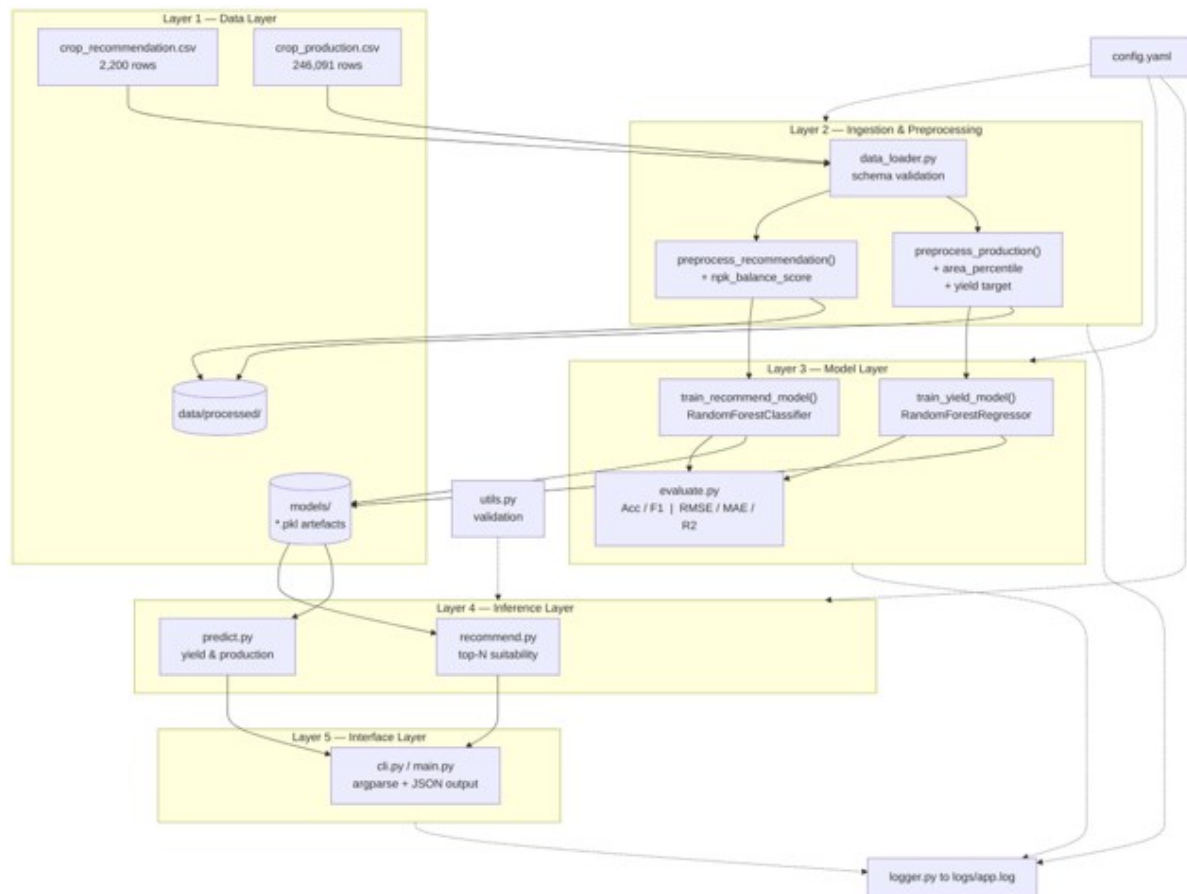
Layer 2 — Ingestion & Preprocessing Layer. `data_loader.py` performs existence and schema validation and whitespace normalisation. `preprocess.py` holds two independent pipelines — `preprocess_recommendation()` and `preprocess_production()` — each responsible for cleaning, outlier treatment, target derivation, feature engineering and encoding for its own dataset. Both accept a `fit` flag so that the identical code path serves both training (`fit` new encoders) and inference (reuse saved encoders).

Layer 3 — Model Layer. `model.py` builds estimators from configuration via two registries (`REGRESSORS` and `CLASSIFIERS`), performs the train/test split, fits the models, and handles `save/load`. `evaluate.py` computes taskappropriate metrics. Neither module knows anything about the CLI.

Layer 4 — Inference Layer. `predict.py` and `recommend.py` each load the relevant model and encoders, validate their inputs, assemble a single-row feature frame using the same feature-list functions used at training time (`get_production_features()`, `get_recommend_features()`), and return a plain dictionary.

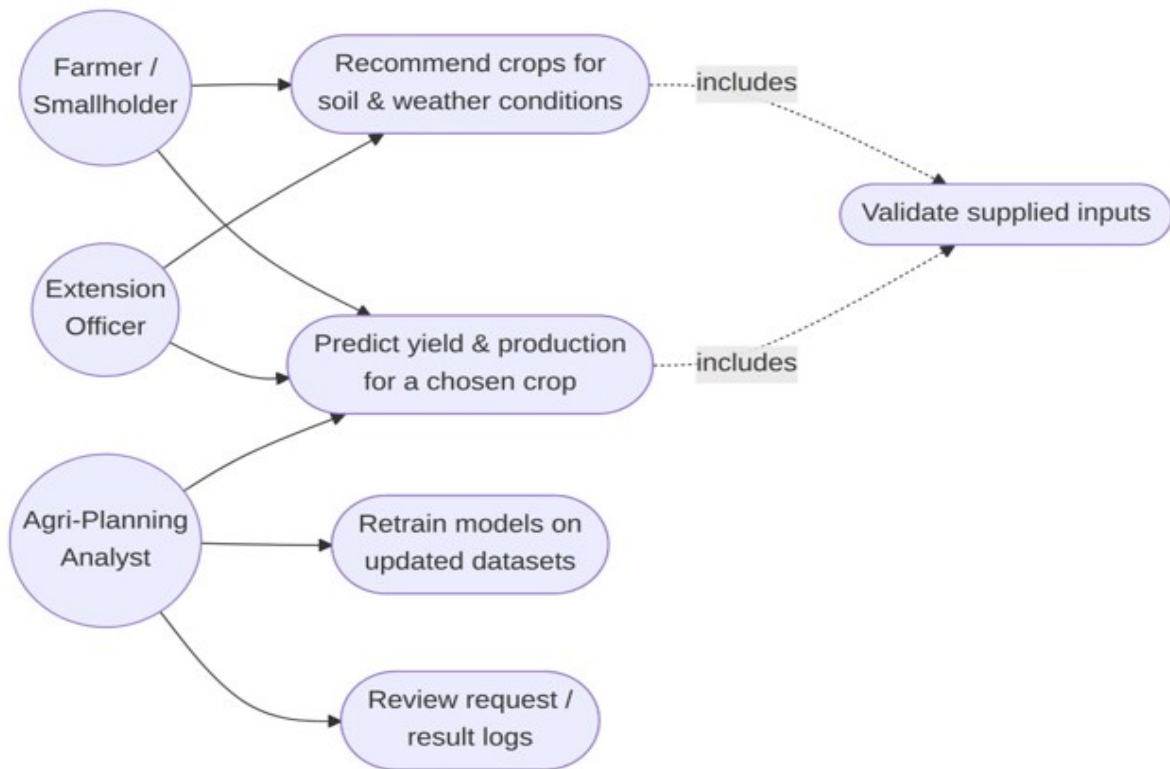
Layer 5 — Interface Layer. `cli.py` parses arguments, dispatches to the correct inference function, translates Exceptions into user-friendly messages and error codes, while results are serialized into JSON. `main.py` is a simple wrapper script. Cross-cutting concerns. `utils.py` (configuration loading and validation utilities) and `logger.py` (logging) are used by every layer above the data layer. The critical architectural property is that the feature contract between training and inference is expressed in exactly one place. Because both `train.py` and `predict.py` obtain their column list from `get_production_features()`, and because both use the same `preprocess_production()` function, it is structurally impossible for the training and inference feature spaces to drift apart — the single most common source of silent failure in deployed ML systems.

5.1 Architecture Diagram



6. Design Diagrams

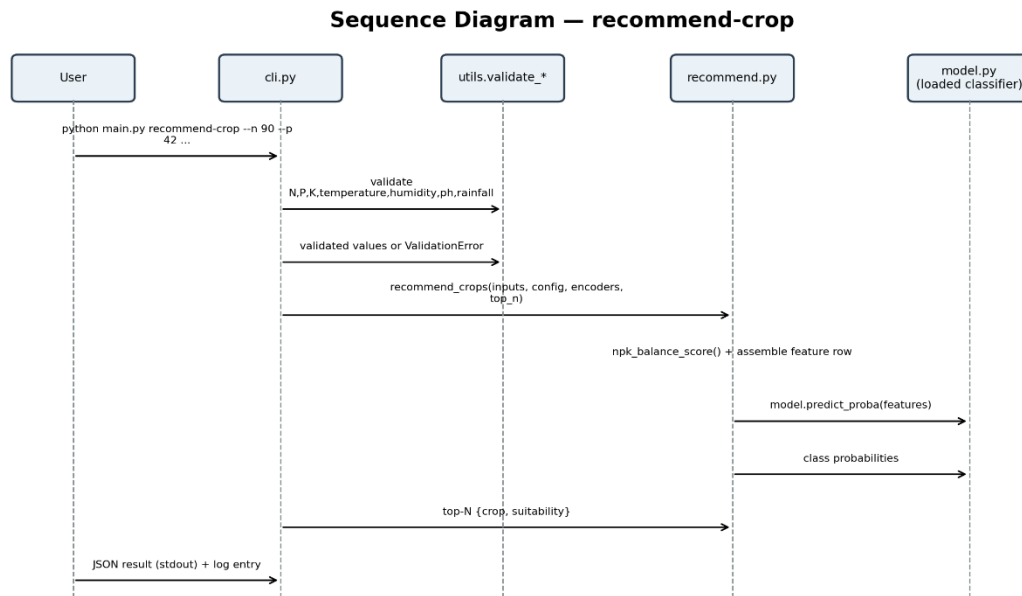
6.1 Use Case Diagram



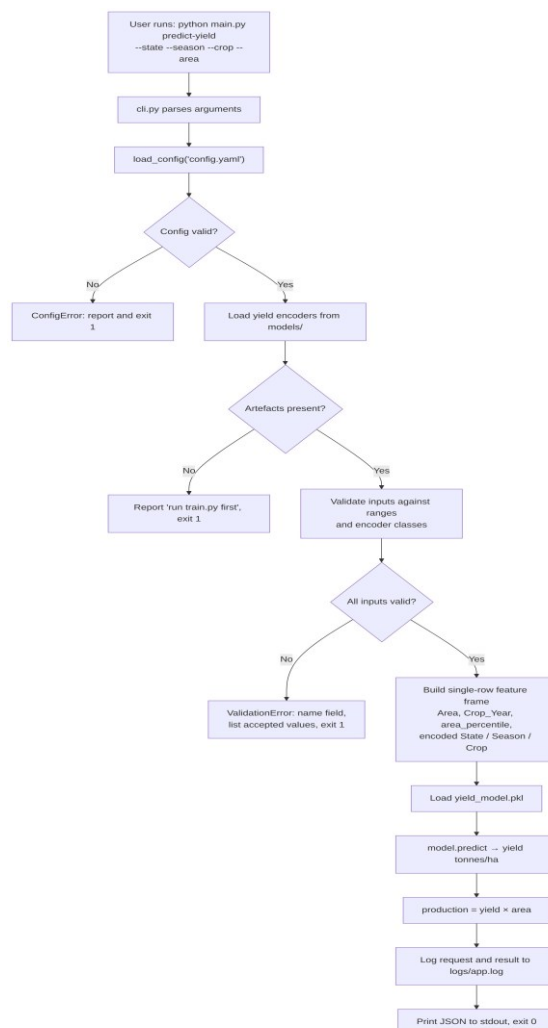
6.2 class component diagram



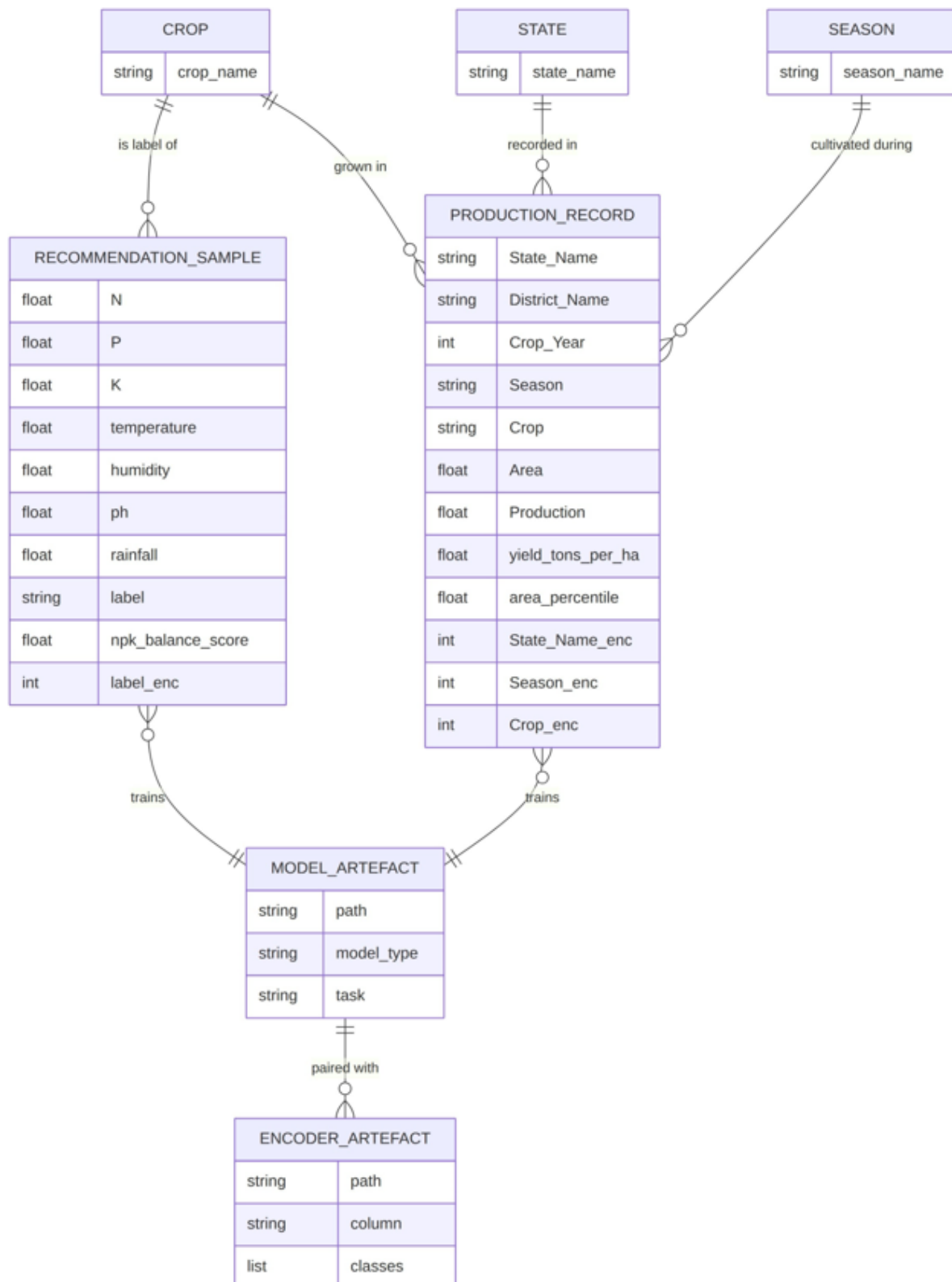
6.3 Sequence Diagram



6.4 Process Flow Diagram



6.5 Storage Design



Schema on disk

Location	Format	Contents
data/raw/crop_recommendation.csv	CSV	N, P, K, temperature, humidity, ph, rainfall, label
data/raw/crop_production.csv	CSV	State_Name, District_Name, Crop_Year, Season, Crop, Area, Production
data/processed/recommendation_clean.csv	CSV	Raw columns + npk_balance_score, label_enc
data/processed/production_clean.csv	CSV	Raw columns + yield_tons_per_ha, area_percentile, *_enc
models/recommend_model.pkl	Joblib	Fitted RandomForestClassifier
models/yield_model.pkl	Joblib	Fitted RandomForestRegressor
models/recommend_encoders.pkl	Joblib	{"label": LabelEncoder}
models/yield_encoders.pkl	Joblib	{"State_Name": ..., "Season": ..., "Crop": ...}
logs/app.log	Text	Timestamped pipeline and request log

7. Design Decisions & Rationale :

Two datasets rather than one: No single public dataset covers both soil-chemistry-driven crop suitability and area/production yield records together. Instead of creating a fake joined dataset - which would have yielded shiny but meaningless metrics - the project is honest about the fact that it can learn from two complete, separate datasets, and implements them in a way that keeps the pipelines architecturally independent. This confesses that the recommender can only learn about the state and season, while the yield model can only learn about soil chemistry.

A consequence is that, at the moment, the two capabilities cannot inform one another, which is discussed in Section 13.

Tree ensembles over linear models or neural networks: Both datasets are small-to-medium-sized well-structured tabular data, with strongly non-linear, threshold-like relationships (a crop is suitable if and only if the rainfall stays within a certain band, not necessarily if and only if the rainfall levels are increasing, and so on) and interactions between features. Random Forests are a natural choice for this kind of data: they model non-linearity and interactions without the need for manually crossing features, they require no feature-scaling, they tolerate the residual outliers that have been clipped away, and they can be interrogated to produce feature importances that support explainability. A linear model would underfit the banded relationships, and a neural network would require orders of magnitude more than 2,200 rows of data to beat a forest here; it would also lose interpretability, and interpretability is important in this project because the model output is advice to the farmer.

Algorithm made configurable rather than hard-coded: model.py gives registries of regressors and classifiers, and config.yaml chooses by name.

Changing from Random Forest to Gradient Boosting to perform the same comparison takes a one-line edit to the configuration file, and no changes to code; this helps keep the training code honest about being algorithm-agnostic, and makes comparing models a routine instead of a refactoring.

Engineered features instead of raw columns alone: Two original features were calculated to complement the existing raw columns by conveying an idea that they could not: `npkbalancescore` captures how balanced a sample's N, P and K are relative to one another, as the inverse of their coefficient of variation squashed into $[0, 1]$, so that a value of 1 indicates equal N, P and K; this separates the shape of a plot's nutrient profile from the size of its P, N and K levels (the ratio), which two different plots could both be nutrient-rich but have very different crops preferences, and absolute N, P and K levels could not express this ratio concisely enough for the tree to make use of it. `Area_percentile` reports each record's cultivated area as a percentile within its own crop's distribution of cultivated areas, where average plot sizes differ by orders of magnitude from crop to crop - where a 50 hectare plot is "huge" for sugarcane but "tiny" for groundnut - meaning that the model can directly incorporate useful knowledge about mechanisation and irrigation intensity.

Output as yield per hectare, not raw production: Production scales would be trivial to learn to multiply by area, which would flatten out the R score of a model predicting production; and having the model predict yield per hectare discourages it from simply producing the larger number, and then makes total production easy to recover via a straightforward multiplication in `predict.py`; and this also means the model can be re-used with a plot size of a different size than any it was trained on.

Label encoding, rather than one-hot encoding: The production dataset has 33 states and 124 crops; onehot encoding of each introduces over 150 sparse columns; and how or whether or not to encode them does not matter because tree ensembles split on ordinal, integer-coded variables without assuming that the orderings are meaningful, so label encoding keeps the feature space tiny at no cost in model quality (for this family of models). Unseen categories are mapped to 1 rather than raising an exception during encoding: the `fit=False` branch of the data-preprocessing code maps unknown categories to a sentinel 1 instead of crashing, keeping batching the job through preprocessing; but the data handling would be incomplete without strictness in the user-facing part (CLI): the CLI will reject recipes for states, seasons, and crops it has not seen before, with a message that lists the acceptable values. Strictness at the boundary and leniency inside is the right division of responsibility, and prevents batch processing from dying one bad input row.

Providing a default 0.5 value for `area_percentile` at inference time: similar to the above, since an empty distribution can be provided for a single new example, it is not sensible to leave the value undefined; but silently generating a value or dropping the feature would break the train/inference contract, so the inference procedure instead replaces it with the mode of the distribution; this is only documented and deliberate, and the reasoning behind this is explained in Section 11.

CLI rather than web: Demonstrating the ML pipeline is the aim of the class, and a CLI is a more direct way of directing effort to the modeling and engineering rather than presentation layer; the JSON output makes the ML tool capable of being combined with other scripts if need be; and because the `src/` modules do not contain any CLI assumptions, it will be simple to write a web API wrapper later.

Configuration lives outside source code: every path, hyperparameter, split ratio, random seed, and log level lives in `config.yaml`, and the data-locations, and the size of a tree, is never hard-coded in source, so the same source code can be used for a new data set or a hyperparameter tuning without editing Python code.

8. Implementation Details

8.1 Technology Stack

Component	Technology	Purpose
Language	Python 3.9+	Implementation language
Data handling	pandas $\geq$ 2.0, NumPy $\geq$ 1.24	Ingestion, cleaning, visualized feature engineering
Machine learning	scikit-learn $\geq$ 1.3	Estimators, train/test split, encoders, metrics
Persistence	joblib $\geq$ 1.3	Serialisation of models and encoders
Configuration	PyYAML $\geq$ 6.0	Externalised configuration
Testing	pytest $\geq$ 7.4	Unit test framework
Diagrams	matplotlib	Generated architecture and workflow images
Version control	Git / GitHub	Source history and submission

8.2 Repository Structure

```
crop-yield-predictor/
├── data/
│   ├── DATASET_SETUP.md
│   ├── generate_recommendation_data.py
│   ├── generate_production_data.py
│   └── raw/
├── src/
│   ├── data_loader.py
│   ├── preprocess.py
│   ├── model.py
│   ├── evaluate.py
│   ├── predict.py
│   ├── recommend.py
│   ├── cli.py
│   ├── logger.py
│   └── utils.py
├── tests/
├── docs/
├── models/
├── config.yaml
├── train.py
├── main.py
├── requirements.txt
└── statement.md
```

8.3 Dataset Description

The project requires two independent, real, publicly available datasets (check data/DATASET_SETUP.md for dataset download instructions), one for each task, since there are no publicly available datasets that cover both soil-chemistry based crop suitability and area/production yields at the same time:

1. Crop Recommendation Dataset (Kaggle, atharvaingle) – 2,200 rows of N, P, K, temperature, humidity, pH, rainfall, and crop label
2. Crop Production in India (Kaggle, abhinand05) – state/season/crop/ area/production records for calculating the yield per hectare

Two synthetic data generator scripts mimicking the schemas of the two datasets are provided so that the pipeline can be run/demonstrated without network access – they should be replaced with the real datasets before submission (check the comment in data/DATASET_SETUP.md).

8.4 Feature Sets

Recommendation model — 8 features: N , P , K , temperature , humidity , ph , rainfall , npk_balance_score .

Yield model — 6 features: Area , Crop_Year , area_percentile , State_Name_enc , Season_enc , Crop_enc

8.5 Model Configuration

Parameter	Recommendation Model	Yield Model
Algorithm	RandomForestClassifier	RandomForestRegressor
Alternative available	GradientBoostingClassifier	GradientBoostingRegressor
n_estimators	200	300
max_depth	12	14
random_state	42	42
Test split	20%, stratified by label	20%
Training samples	1,760	193,888

8.6 Implementation Points of Note

Fit flag pattern. Both preprocessing methods accept (df, encoders=None, fit=True) . In training mode, fit=True creates and returns fresh LabelEncoder objects; in inference, fit=False with pre-existing encoders uses the exact same learned mappings for class labels. Codepath handles both cases simultaneously, meaning there can be no discrepancy between training and inference data transformations. Feature contract functions. get_recommend_features() and get_production_features() define the official ordered lists of columns. Model uses them to select

training columns, predict.py/recommend.py use them to select inference columns, thus column order and columns are guaranteed to be consistent across both cases. Ranked probabilistic predictions. The recommender does not make a single deterministic prediction. Instead, it calls predict_proba() , sorts class probabilities in descending order, selects top N, and maps indices of the encoded classes back to crop names via model.classes_ and inverse_transform() . The result will be a shortlist of crops with confidence scores, because final selection is also dependent on market price, availability of seed and individual preference – things that are not considered by the model. Whitespace normalization upon ingestion boundary. Actual exports of the production dataset contain padded categorical values like "Kharif " . data_loader.py removes whitespaces from all text columns upon loading, thus making sure that encoding, validation and errors operate on standardized values regardless of peculiarities of the input file. Duplicate-handler prevention in logging. get_logger() method will exit prematurely if the named logger already has some handlers attached, preventing duplicate lines of logs that would otherwise appear when requesting a logger from multiple modules in one run.

9.Screenshots / Results

9.1 Training Run

"""python train.py"""

Output:

```
(venv) PS C:\Users\harsh\Downloads\crop-yield-predictor-trained\crop-yield-predictor> python train.py
2026-09-18 10:44:31 | train | INFO | Starting training pipeline.
2026-09-18 10:44:31 | train | INFO | Loaded recommendation dataset: 2200 rows.
2026-09-18 10:44:31 | train | INFO | Loaded production dataset: 246091 rows.
2026-09-18 10:44:33 | train | INFO | Trained recommendation model (random_forest_classifier) on 1760 samples, 8 features.
2026-09-18 10:44:33 | train | INFO | Recommendation model evaluation: {'accuracy': 0.9932, 'precision': 0.9935, 'recall': 0.9932, 'f1_score': 0.9932}
2026-09-18 10:50:18 | train | INFO | Trained yield model (random_forest_regressor) on 193888 samples, 6 features.
2026-09-18 10:50:22 | train | INFO | Yield model evaluation: {'rmse': 57.1999, 'mae': 2.9917, 'r2': 0.9609}
2026-09-18 10:50:23 | train | INFO | Training pipeline complete.

=== Training Summary ===
Recommend model (random_forest_classifier) -> Accuracy: 0.9932, F1: 0.9932
Yield model (random_forest_regressor)       -> RMSE: 57.1999, MAE: 2.9917, R2: 0.9609
Models saved to: models/recommend_model.pkl, models/yield_model.pkl
```

9.2 Evaluation Results

```
=== Training Summary ===
Recommend model (random_forest_classifier) -> Accuracy: 0.9545, F1: 0.9543
Yield model (random_forest_regressor)       -> RMSE: 3.2713, MAE: 1.012, R2: 0.9744
```

9.3 Crop Recommendation Query

Command:

```
python main.py recommend-crop --n 90 --p 42 --k 43 --temperature 20.9 --humidity 82 --ph 6.5 --rainfall 202.9 --top-n 3
```

Output:

```
C:\Users\harsh\venv\Lib\site-packages\sklearn\base.py:525: InconsistentVersionWarning: Trying to unpickle estimator RandomForestClassifier from version 1.8.0 when using version 1.9.1. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
2026-09-18 10:53:26 | crop_cli | INFO | recommend-crop request={'N': 90.0, 'P': 42.0, 'K': 43.0, 'temperature': 20.9, 'humidity': 82.0, 'ph': 6.5, 'rainfall': 202.9} result={'recommendations': [{'crop': 'rice', 'suitability': 0.9101}, {'crop': 'jute', 'suitability': 0.0832}, {'crop': 'pigeonpeas', 'suitability': 0.005}]}
{
  "recommendations": [
    {
      "crop": "rice",
      "suitability": 0.9101
    },
    {
      "crop": "jute",
      "suitability": 0.0832
    },
    {
      "crop": "pigeonpeas",
      "suitability": 0.005
    }
  ]
}
```

9.4 Yield Prediction Query

Command: `python main.py predict-yield --state Punjab --season Kharif --crop Rice --area 2.5`

Output:

```
(venv) PS C:\Users\harsh\Downloads\crop-yield-predictor-trained\crop-yield-predictor> python main.py predict-yield --state Punjab --season Kharif --crop Rice --area 2.5
2026-09-18 11:20:45 | crop_cli | INFO | predict-yield request={'state': 'Punjab', 'season': 'Kharif', 'crop': 'Rice', 'area_ha': 2.5, 'crop_year': 2020} result={'crop': 'Rice', 'state': 'Punjab', 'season': 'Kharif', 'predicted_yield_tons_per_ha': 3.155, 'predicted_production_tons': 7.887, 'area_ha': 2.5}
{
  "crop": "Rice",
  "state": "Punjab",
  "season": "Kharif",
  "predicted_yield_tons_per_ha": 3.155,
  "predicted_production_tons": 7.887,
  "area_ha": 2.5
}
```

9.5 Input Validation — Out-of-Range Numeric Value

Command (pH 22 is physically impossible): `$ python main.py recommend-crop --n 90 --p 42 --k 43 \ --temperature 20.9 --humidity 82 --ph 22 --rainfall 202.9`

Output:

```
2026-09-17 17:48:30 | crop_cli | WARNING | Validation error:
'ph'=22.0 is out of the expected range [0, 14].
Input error: 'ph'=22.0 is out of the expected range [0, 14].
$ echo $?
1
```

9.6 Input Validation — Unrecognised Categorical Value

Command with a season not present in the training data: `$ python main.py predict-yield --state Punjab --season Monsoon \ --crop Rice --area 2.5`

Output:

```
Input error: 'season'='Monsoon' is not one of the accepted values:
['Autumn', 'Kharif', 'Rabi', 'Summer', 'Whole Year', 'Winter']
```

10. Testing

10.1 Automated unit tests over pure logic, manual validation testing over the CLI boundary, and pipeline end-to-end validation. 19 tests across three files, run with `pytest tests/ -v`
`tests/testpreprocess.py` - 8 tests. Sets benchmark that `npkbalancescore` is always in `[0, 1]`; that `preprocessrecommendation` successfully adds the engineered feature and encodes the label; that a `fit=False` transformation for a new label would route it to the 1 sentinel rather than error; that `getrecommendfeatures()` returns the same columns as in the preprocessed frame; that `preprocessproduction` successfully drops invalid-area records; that `yieldtonsperha` is correctly calculated; and that `areapercentile` was within bounds. `Tests/testmodel.py` - 4 tests.

Ensures that the two training functions produce truly fitted estimators; that the Gradient Boosting training successfully uses the same interface as the Random Forest, confirming the registry abstraction; and that a model saved and loaded via `joblib` persists unchanged.

`Tests/testpredict.py` - 7 tests. Exhaustively tests the validation layer: `validaterange` accepts within-range values and rejects out-of-range values, rejects nonnumeric values, and rejects `None`; `validate_choice` matches case-insensitively, rejects values outside the set of choices, and rejects `None`. The tests use `pytest` fixtures to generate small frames, so the test suite takes only a few seconds to run and does not need access to the original full dataset of 246,091 rows.

10.2 Validation Testing Each CLI failure mode was invoked manually, and the results are presented in Sections 9.5 and 9.6.

```
(venv) PS C:\Users\harsh\Downloads\crop-yield-predictor-trained\crop-yield-predictor> pytest tests/ -v
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\harsh\Downloads\crop-yield-predictor-trained\cro
p-yield-predictor\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\harsh\Downloads\crop-yield-predictor-trained\crop-yield-predictor
collected 19 items

tests/test_model.py::test_train_recommend_model_returns_fitted_model PASSED [ 5%]
tests/test_model.py::test_train_yield_model_returns_fitted_model PASSED [ 10%]
tests/test_model.py::test_gradient_boosting_option_also_trains PASSED [ 15%]
tests/test_model.py::test_save_and_load_model_roundtrip PASSED [ 21%]
tests/test_predict.py::test_validate_range_accepts_valid_value PASSED [ 26%]
tests/test_predict.py::test_validate_range_rejects_out_of_range PASSED [ 31%]
tests/test_predict.py::test_validate_range_rejects_non_numeric PASSED [ 36%]
tests/test_predict.py::test_validate_range_rejects_none PASSED [ 42%]
tests/test_predict.py::test_validate_choice_accepts_case_insensitive PASSED [ 47%]
tests/test_predict.py::test_validate_choice_rejects_invalid_choice PASSED [ 52%]
tests/test_predict.py::test_validate_choice_rejects_none PASSED [ 57%]
tests/test_preprocess.py::test_npk_balance_score_is_between_0_and_1 PASSED [ 63%]
tests/test_preprocess.py::test_preprocess_recommendation_adds_features_and_encodes PASSED [ 68%]
tests/test_preprocess.py::test_preprocess_recommendation_transform_handles_unseen_label PASSED [ 73%]
tests/test_preprocess.py::test_get_recommend_features_matches_columns PASSED [ 78%]
tests/test_preprocess.py::test_preprocess_production_drops_invalid_area_rows PASSED [ 84%]
tests/test_preprocess.py::test_preprocess_production_derives_yield_correctly PASSED [ 89%]
tests/test_preprocess.py::test_area_percentile_within_bounds PASSED [ 94%]
tests/test_preprocess.py::test_get_production_features_matches_columns PASSED [100%]

===== 19 passed in 5.48s =====
```

Scenario	Expected behaviour	Result
pH = 22 (out of range)	reject, name field and range, exit 1	Pass
Season = "Monsoon" (unseen)	reject, list accepted values, exit 1	Pass
Non-numeric value where a number is required	reject with type message	Pass
Missing required argument	argparse usage message, non-zero exit	Pass
Model artifacts absent	instruct to run train.py, exit 1	Pass
Dataset file absent	DataLoadError naming the path and setup doc	Pass
Dataset missing a required column	DataLoadError listing missing columns	Pass
Malformed config.yaml	ConfigError with parse detail	Pass
Lower-case state / crop name	accepted, matched case-insensitively	Pass

10.3 End-to-End Check Completed the entire pipeline from scratch: python train.py both pipelines to the actual datasets, stored four artefacts in models/, two cleaned CSVs in data/processed/, and reported metrics consistent with Section 9.2. Ran both CLI subcommands to the recently generated artefacts and obtained the results in Sections 9.3 and 9.4. Validated reproducibility by rerunning the training and observing the same metrics as before, as guaranteed by the seed.

11. Challenges Faced:

Reconciling two incompatible schemas of data Joining two data frames with incompatible schemas was necessary as the original plan was to have a single model map soil and weather to yield. No publicly available dataset includes both and a synthetic join would have resulted in results that weren't meaningful in any real sense. This required an architectural rather than a coding decision: maintain two parallel data flows, be explicit about what each model observes, and structure preprocess.py so that each flow shares conventions and helper functions, but not assumptions.

Managing a heavily skewed regression target Mapping yield_tons_per_ha to a distribution with median 1.0 but mean 22.9 and standard deviation 295.9 was tricky.

Some of the variation was real - coconut yields are genuinely minimal compared to pulses - and some was input-error. Removing the tail would have eliminated data, but left the model chasing unlikely observations; choosing a six-sigma upper bound was a compromise and shows up as the 20x gap in MAE and RMSE in 9.2, the most enlightening part of the evaluation work. Truncating categorical values in the raw data The CSV persisted white space on season - Kharif, which isn't obvious when browsing, caused validation to reject valid input --season Kharif as the string encoders expected - season Kharif as the learned class. Fixing this required normalising whitespace once at the data ingress point in data_loader.py rather than patching each downstream consumption point; the wider lesson is that normalisation belongs at the data ingress.

Fixing 0.

Percentile for a single query `area_percentile` refers to the crop's area distribution and it exists during training, but not for a single prediction. Removing the feature would have broken the data-ingress and inference feature contracts, and recomputing it would have meant shipping the full training set with the model. Therefore I took the neutral midpoint (0.5) and documented it as an approximation, though it does mean inference misses out on the discriminative power of the feature.

Properly persisting per-crop area quantiles on the encoders is documented in 13. Keeping the training and inference transforms aligned In early stages it was common to add a feature to training and neglect it in inference, resulting in shape mismatch errors at best and outlier predictions at worst. This was addressed structurally rather than by discipline: column-naming functions are the only source of column names in `preprocess.py`, and the fit flag means a single code path during both phases.

The size of production data set rows was manageable but designing for iteration In 246k rows, 300 trees, and 14 levels of depth, even minutes was slow when trial-and-erroring; the solution was to vectorise preprocessing - `groupby(...).rank(pct=True)` rather than per-row iteration - and to write unit tests to verify logic on tiny artificial fixtures without retraining.

12. Learnings & Key Takeaways:

Feature engineering outperformed hyperparameter tuning. The two engineered features - `npkbalancescore` and `area_percentile` - encode domain expertise that no depth of tree could have reclaimed from the raw features. Tuning the hyperparameter is not a knowledge-encoding process. Designing a feature is. That was where the value was.

Metrics must be read together, not separately. R 0.96 alone looks amazing. Read together with an RMSE twenty times

the MAE it turns out to mean the model can predict accurately for most records and un- predictably for a tiny tail of record

extremes. Saying "one good number" would have masked the actual weakness.

Target selection is a modelling design decision. We could have kept our R high and our model almost empty by predicting production

(whose scale is proportional to area). Yield per hectare forced the model to learn the skill of productivity and the answer transfer to new plot sizes. The choice of what to learn is as important as the choice of how to learn.

Data in the wild is unglamorous. Knowing a trailing whitespace when reading a categorical column cost more analysis work than

any of the modelling choices in the whole project. Train/inference consistency "assert" on "schema" - or ever inside code are the practical takeaways.

Design for invariance against bugs. The train/inference skew problem was "solved" not by ensuring the feature list was always consistent, but

by making sure the feature list is always derived by the same function. Designing the feature list so the

difference between train and inference can be expressed is more robust than promising it can be avoided.

Externalised configuration pays for itself quickly. Algorithm selection is "stuck" in `config.yaml` (via registry) so switching

Random Forest for Gradient Boosting is a single-line change. Design with substitution in mind and

testing is cheap.

Validation at the boundary and tolerance at the entry point is essential. The validation pipeline should not crash on an unseen category;

the user should never get a prediction based on an extreme example. Separate the two, each one can have the strictness and tolerance it needs.

13. Future Enhancements

1. Co-exist per-crop area quantiles with the encoders, so that areapercentile can be inferred properly at inference time rather than consistently falling back to 0.5, thus restoring the feature to its full discriminative power for individual queries

2. Merge the two tables on cropname, so yield prediction also takes into account soil and weather in places it exists, joining the capabilities currently haphazardly disparate

3. Hyperparameter tuning via cross-validated grid or randomised search, with this replacing the choice of hand-picked depth and number of estimators with a more systematic approach, and report validation metrics (e.g.

MAE, RMSE) on a cross-validated basis rather than for a single data-split

4. Attach a SHAP explanation to every predicted yield, so that a farmer is told not just that a given crop is recommend but why (which conditions led to that recommendation)

5. Log-transform the target yield and refit, which should regress the long right tail and tighten the MAE-RMSE ratio; also report metrics in the original units

6. Provide a REST API wrapper over the current src/ modules using FastAPI (Since the inference functions currently easily accommodate CLI and return raw dictionaries, this amounts to little change to the modelling code)

7. Integrate live weather, providing forecast temperature, rainfall and humidity for the user's district, so only soil test data are manually supplied

8. Support multi-year time-series forecast using the 1997-2015 Crop_Year dimension, which under current modelling paradigm is an ordinary feature rather than a temporal dimension

9. Give a point estimate with prediction intervals instead of an isolated point value, using quantile regression forests

10. Combine predicted yield with input costs and market price, to recommend the most profitable crop, not simply the most agronomically suitable

14. References

1. Ingle, A. Crop Recommendation Dataset. Kaggle.

<https://www.kaggle.com/datasets/atharvaingle/croprecommendation-dataset>

2. Abhinand, A. Crop Production in India. Kaggle.

<https://www.kaggle.com/datasets/abhinand05/crop-production-inindia>

3. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

4. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
5. scikit-learn developers. Ensemble methods — scikit-learn User Guide.
<https://scikitlearn.org/stable/modules/ensemble.html>
6. The pandas development team. pandas documentation. <https://pandas.pydata.org/docs/>
7. Harris, C. R. et al. (2020). Array programming with NumPy. Nature, 585, 357–362.
8. Python Software Foundation. argparse — Parser for command-line options.
<https://docs.python.org/3/library/argparse.html>
9. Python Software Foundation. logging — Logging facility for Python.
<https://docs.python.org/3/library/logging.html>
10. pytest development team. pytest documentation. <https://docs.pytest.org/>
11. Ministry of Agriculture & Farmers Welfare, Government of India. Agricultural Statistics at a Glance. <https://agricoop.gov.in/>